

RETO V

Diseñe un programa que dados dos números muestre el resultado de la suma, resta, multiplicación, división, raíz cuadrada y potenciación

Codigo en C++:

```
#include <iostream>
#include <cmath>
#include <string>
#include <algorithm>
#include <stdexcept>

using namespace std;

int main() {
//Variables
    double numero_1, numero_2, resultado, Indice, exponente;
    string operacion;

    cout << "Bienvenido a su calculadora" << endl;
//Identificador de errores
    try {
        // Preguntar operación
        cout << "¿Qué operacion desea realizar?" << endl;

        cout << "Suma, Resta, Multiplicacion, Division, Potencia, Factorial, Valor absoluto, Raiz" << endl;

        getline(cin, operacion);

        // Convertidor a minusculas
```

```
transform(operacion.begin(), operacion.end(), operacion.begin(), ::tolower);
```

```
// Suma
```

```
if (operacion == "suma" || operacion == "+") {  
    cout << "numero_1: ";  
    if (!(cin >> numero_1)) throw invalid_argument("Debe ingresar un numero  
valido");  
    cout << "numero_2: ";  
    if (!(cin >> numero_2)) throw invalid_argument("Debe ingresar un numero  
valido");  
  
    resultado = numero_1 + numero_2;  
    cout << "Resultado: " << resultado << endl;  
}
```

```
//Resta
```

```
else if (operacion == "resta" || operacion == "-") {  
    cout << "numero_1: ";  
    if (!(cin >> numero_1)) throw invalid_argument("Debe ingresar un numero  
valido");  
    cout << "numero_2: ";  
    if (!(cin >> numero_2)) throw invalid_argument("Debe ingresar un numero  
valido");  
  
    resultado = numero_1 - numero_2;  
    cout << "Resultado: " << resultado << endl;  
}
```

```

//Multiplicacion
else if (operacion == "multiplicacion" || operacion == "**") {
    cout << "numero_1: ";
    if (!(cin >> numero_1)) throw invalid_argument("Debe ingresar un numero
valido");
    cout << "numero_2: ";
    if (!(cin >> numero_2)) throw invalid_argument("Debe ingresar un numero
valido");

    resultado = numero_1 * numero_2;
    cout << "Resultado: " << resultado << endl;
}

//Division
else if (operacion == "division" || operacion == "/") {
    cout << "numero_1: ";
    if (!(cin >> numero_1)) throw invalid_argument("Debe ingresar un numero
valido");
    cout << "numero_2: ";
    if (!(cin >> numero_2)) throw invalid_argument("Debe ingresar un numero
valido");

    if (numero_2 == 0) throw invalid_argument("No se puede dividir entre 0");

    resultado = numero_1 / numero_2;
    cout << "Resultado: " << resultado << endl;
}

```

```

//Potencia
else if (operacion == "potencia" || operacion == "**") {
    cout << "numero_1: ";
    if (!(cin >> numero_1)) throw invalid_argument("Debe ingresar un numero
valido");
    cout << "numero_2: ";
    if (!(cin >> numero_2)) throw invalid_argument("Debe ingresar un numero
valido");

    resultado = pow(numero_1, numero_2);
    cout << "Resultado: " << resultado << endl;
}

//Factorial
else if (operacion == "factorial" || operacion == "!") {
    int n;
    cout << "Ingrese un numero entero positivo: ";
    if (!(cin >> n)) throw invalid_argument("Debe ingresar un numero valido");

    if (n < 0) throw invalid_argument("El factorial no existe para numeros
negativos");

    long long factorial = 1;
    for (int i = 1; i <= n; i++) factorial *= i;

    cout << "Resultado: " << factorial << endl;
}

```

```

//Valor absoluto
else if (operacion == "valor absoluto" || operacion == "||") {
    cout << "numero: ";
    if (!(cin >> numero_1)) throw invalid_argument("Debe ingresar un numero
valido");

    resultado = fabs(numero_1);
    cout << "Resultado: " << resultado << endl;
}

//Raiz
else if (operacion == "raiz") {
    cout << "Indice de la raiz: ";
    if (!(cin >> Indice)) throw invalid_argument("Debe ingresar un numero
valido");
    cout << "Numero radicado: ";
    if (!(cin >> numero_1)) throw invalid_argument("Debe ingresar un numero
valido");

    if (Indice == 0) throw invalid_argument("El indice de la raiz no puede ser
0");

    exponente = 1.0 / Indice; // división decimal correcta
    resultado = pow(numero_1, exponente);
    cout << "Resultado: " << resultado << endl;
}

else {

```

```
        throw invalid_argument("Operacion no reconocida");
    }
}

catch (invalid_argument& e) {
    cout << "Error: " << e.what() << endl;
}

return 0;
}
```

Diagrama de flujo:







